



SISTEMAS OPERATIVOS - LABORATORIO
12 de Junio de 2026

Nombre _____ DNI _____
Apellidos _____ Grupo _____

INSTRUCCIONES

- Se debe entregar un **archivo** que incluya una carpeta por apartado con el código correspondiente a dicho apartado (ej: 1A, 1B, etc.), para así garantizar la evaluación independiente de cada uno.
- Si el código no compila o su ejecución produce un error grave la puntuación de ese apartado será 0.

Nota: se recomienda crear el **archivo** por línea de comandos. Por ejemplo, si todo el contenido lo tenemos en un subdirectorío examen/ dentro del directorio de trabajo actual podemos usar: `tar cvzf examen.tar.gz examen/`

Cuestión 1. (5 puntos) Tomando como referencia los ejercicios 1 y 3 de la práctica 4, diseñe un programa que se comporte del siguiente modo:

- a) **(2 puntos)** El programa recibe **como argumentos de la línea de comandos** una serie de comandos Linux a ejecutar. Para cada uno de estos comandos el programa deberá:
- i) Crear un proceso que ejecute el programa `/bin/bash` con la opción `-c`, pasando como argumento de `-c` la cadena del comando a ejecutar. Es decir, se usará `bash` para parsear e interpretar el comando.
 - ii) Mostrar una cadena por el terminal de salida que muestre el comando ejecutado y el PID del proceso que lo ejecuta (el PID del proceso que ejecuta el `bash` que interpreta el comando).

```
$ ./cuestion1 "sleep 3" "ls -l | grep cuestion1" "pkill -KILL sleep"
[214628] sleep 3
[214629] ls -l | grep cuestion1
[214630] pkill -KILL sleep
-rwxrwxr-x 1 usuario usuario 21688 Jun 1 12:13 cuestion1
-rw-rw-r-- 1 usuario usuario 1381 Jun 1 12:13 cuestion1.c
```

- b) **(2 puntos)** Usando **getopt**, añadir al programa la opción de tratar el flag `-w`. Si se invoca el programa con esta opción, una vez lanzados a ejecución todos los procesos que ejecutan los comandos, el programa se quedará esperando a que terminen los procesos, y según vayan terminando mostrará una cadena informando del pid del proceso y el motivo de su finalización. Si el proceso fue terminado por una señal indicará el número de señal. Si el proceso terminó normalmente mostrará el código de terminación que devolvió el proceso al sistema.

```
$ ./cuestion1 -w "sleep 3" "ls -l | grep cuestion1" "pkill -KILL sleep"
[1894520] sleep 3
[1894521] ls -l | grep cuestion1
[1894522] pkill -KILL sleep
-rwxrwxr-x 1 usuario usuario 21840 Jun 1 12:13 cuestion1
-rw-rw-r-- 1 usuario usuario 1487 Jun 1 12:13 cuestion1.c
process 1894521 exited with status 0
process 1894520 terminated by signal 9
process 1894522 exited with status 0
```

- c) **(1 punto)** Modificar el programa de modo que si se pasa la opción `-i`:
- i) El programa deberá **ignorar** la señal `SIGINT` (no tratarla con un handler sino ignorarla).
 - ii) Los procesos que ejecutan los comandos deberán **ignorar** tanto `SIGINT` como `SIGTERM`.

```
$ ./cuestion1 -wi "sleep 3" "ls -l | grep cuestion1" "pkill -TERM sleep"
[215244] sleep 3
[215245] ls -l | grep cuestion1
[215246] pkill -TERM sleep
-rwxrwxr-x 1 usuario usuario 21840 Jun 13 18:44 cuestion1
-rw-rw-r-- 1 usuario usuario 1487 Jun 13 18:44 cuestion1.c
process 215245 exited with status 0
process 215246 exited with status 0
process 215244 exited with status 0
```

Cuestión 2. (5 puntos) Partiendo de tu solución de la práctica 5, modifica el programa para que:

- (1 punto)** A cada hilo cliente se le pasará un parámetro adicional *rounds*. El valor de este parámetro se leerá del fichero de entrada igual que el flag que indica si es vip o no vip, y será un número entero. Se modificará el código del cliente de forma que, una vez dentro de la discoteca, bailará tantas veces como indique su parámetro *rounds*.
- (1.5 puntos)** En cada ronda, el cliente deberá obtener una bebida y beberla antes de bailar. Si no hay bebidas disponibles avisará al camarero (ver apartado c) y se quedará esperando hasta que el camarero reponga las bebidas. La simulación de beber se hará con un sleep aleatorio de 1 a 3 segundos. Múltiples clientes pueden estar bebiendo a la vez, y a la vez que otros están bailando.
- (1.5 puntos)** El camarero será simulado con un nuevo hilo POSIX, creado al iniciar el programa. Estará siempre esperando a que los clientes le avisen si se han acabado las bebidas. Cuando sea avisado pondrá 5 bebidas en el mostrador y volverá a esperar a que algún cliente le diga que se han agotado.
- (1 punto)** Cuando hayan terminado todos los hilos cliente, el programa debe notificar al camarero de que debe finalizar y luego esperar a que el hilo del camarero termine, para poder hacer un cierre ordenado del programa.

La descripción anterior indica cómo se valorará cada parte, pero hay que tener en cuenta que los apartados b y c dependen el uno del otro, por tanto es recomendable implementarlas juntas.

NOTA: el profesor podrá descontar puntos si las condiciones de entrada de los clientes indicadas en el enunciado de la práctica 5 no se cumplen correctamente en el código entregado.

<pre>\$ cat ejemplo.txt 5 0 1 0 2 1 3 0 1 1 1 \$./disco ejemplo.txt client 0 is not vip client 1 is not vip client 2 is vip client 3 is not vip Starting client 0 (dancing for 1 rounds) Starting client 1 (dancing for 2 rounds) Starting client 3 (dancing for 1 rounds) Client 0 (not vip) entering the disco Starting waiter client 4 is vip Client 0 (not vip) calling waiter Starting client 2 (dancing for 3 rounds) Client 1 (not vip) entering the disco Client 1 (not vip) calling waiter Client 3 (not vip) entering the disco Client 3 (not vip) calling waiter Starting client 4 (dancing for 1 rounds) Waiter serving 5 drinks Client 0 (not vip) getting a drink Client 0 (not vip) drinking Client 2 (vip) entering the disco Client 2 (vip) getting a drink Client 2 (vip) drinking Client 1 (not vip) getting a drink Client 1 (not vip) drinking Client 4 (vip) entering the disco Client 4 (vip) getting a drink Client 4 (vip) drinking Client 3 (not vip) getting a drink Client 3 (not vip) drinking Client 1 (not vip) dancing in disco Client 2 (vip) dancing in disco Client 0 (not vip) dancing in disco Client 4 (vip) dancing in disco Client 3 (not vip) dancing in disco</pre>	<pre>Client 0 (not vip) exits the disco Client 4 (vip) exits the disco Client 1 (not vip) calling waiter Waiter serving 5 drinks Client 1 (not vip) getting a drink Client 1 (not vip) drinking Client 2 (vip) getting a drink Client 2 (vip) drinking Client 3 (not vip) exits the disco Client 2 (vip) dancing in disco Client 1 (not vip) dancing in disco Client 1 (not vip) exits the disco Client 2 (vip) getting a drink Client 2 (vip) drinking Client 2 (vip) dancing in disco Client 2 (vip) exits the disco</pre>
--	--